

Start Here - Daily Keys

Ctrl+Shift+P / F1	Command Palette: find any command
Ctrl+P	Quick Open file by name
Ctrl+,	Settings UI
Ctrl+K Ctrl+S	Keyboard Shortcuts editor
Ctrl+B	Toggle sidebar
Ctrl+Shift+E	Explorer
Ctrl+Shift+G	Source Control
Ctrl+Shift+U	Output panel

Navigation / Editor Control

Ctrl+G	Go to line
Ctrl+T	Show all symbols
Ctrl+Shift+O	Symbols in current file
Alt+Left/Right	Go back / forward
Ctrl+I	Split editor
Ctrl+J/2/3	Focus editor group
Ctrl+K P	Copy active file path
Ctrl+K R	Reveal active file in Explorer

Config Editing Moves

Alt+Z	Word wrap long config lines
Ctrl+/	Toggle line comment
Shift+Alt+A	Block comment
Alt+Up/Down	Move line up/down
Shift+Alt+Up/Down	Copy line up/down
Ctrl+Shift+K	Delete line
Ctrl+Enter	Insert line below
Ctrl+]/ Ctrl+[	Indent / outdent
Ctrl+K Ctrl+X	Trim trailing whitespace
Ctrl+K M	Change language mode

Search Like an Admin

Ctrl+F / Ctrl+H	Find / replace in file
Ctrl+Shift+F	Search repo/workspace
Ctrl+Shift+H	Replace across files
Alt+CW/R	Case / word / regex in find
Include	**/*.yaml, **/*.sh, **/*.ps1, **/*.service, **/*.repo, **/*.ks
Exclude	**/.git/**, **/vendor/**, **/*.log, **/node_modules/**
Rule	Search first. Replace second. Review diff third.

Multi-Cursor Surgery

Alt+Click	Add cursor at exact spot
Ctrl+D	Add next matching selection
Ctrl+Shift+L	Select all occurrences of selection
Ctrl+F2	Select all occurrences of current word
Ctrl+U	Undo last cursor operation
Shift+Alt+I	Cursor at end of each selected line
Use case	Bulk-edit hostnames, repo stanzas, unit names, CIDR/IP lists, YAML keys.

Diff / Review

SCM view	Click changed file to open side-by-side diff
code --diff	Compare two files from terminal
Timeline	Per-file history and local changes
Whitespace	Do not ignore whitespace for YAML, Python, shell, unit files.
Before commit	Read every changed hunk; remove debug junk and secrets.

Git / Source Control Workflow

Open folder	Open the repo root, not a single file
Review first	Click changed file -> inspect diff
Stage carefully	Use + per file; stage hunks/lines from diff when useful
Commit	Short imperative subject: 'Fix RHEL10 repo template'
Sync/push	Prefer explicit pull/fetch/push on controlled networks
Conflict rule	Resolve in editor, then re-run validation before commit.

Terminal Habits

Ctrl+`	Toggle terminal at workspace root
Ctrl+Shift+`	New terminal
Ctrl+Shift+C	External terminal
Shells	PowerShell for Windows admin; Git Bash for Unix-like Git/SSH; WSL only if approved
Split by task	1: git, 2: ssh, 3: test/lint, 4: notes
State check	Run git status before and after risky operations.

Useful CLI

code .	Open current folder
code -r .	Reuse current VS Code window
code --diff a b	Open file diff
code --wait file	Use VS Code as blocking editor for a command
git status	First and last command of the change
git diff	Audit unstaged work

Your Work Pattern

Golden path	Edit locally in VS Code -> commit to internal Git/Gitea -> deploy/test on Linux target.
Track in repo	Kickstarts, PXE menus, Ansible/playbooks, .repo files, systemd units, HTCondor configs, scripts.
Avoid	Silent hand edits on production /etc. If emergency-editing live, backport the exact change into Git.
Change record	Commit message should say what changed and why; ticket/SR if your shop uses them.

Pre-Commit Safety Gate

1	git status - know every changed file
2	git diff - read every hunk
3	Run syntax/lint checks available in the air-gapped network
4	Search for passwords, tokens, keys, host-specific junk
5	Confirm CRLF vs LF and final newline
6	Commit small; push only after local validation

Validation Commands

Bash	bash -n script.sh; shellcheck script.sh if available
PowerShell	pwsh -NoProfile -File .\script.ps1; PSScriptAnalyzer if available
YAML	yamllint file.yml or ansible-playbook --syntax-check
systemd	systemd-analyze verify unit.service
Kickstart	Validate generated files and test in a disposable VM before fleet use.

Remote SSH - Useful but Deliberate

Best use	Non-prod troubleshooting, reading logs, editing scratch files, working in a dev VM.
Caution	Remote-SSH installs/uses a VS Code Server under the user account; may be unwanted on hardened hosts.
Prod stance	Use VS Code locally for repo changes; deploy through approved scripts/tools; SSH for observation.
When it fails	Check plain ssh first, host key, shell startup noise, quota, noexec home/tmp, firewall/proxy.

Recommended Settings JSON

Open	Ctrl+Shift+P -> Preferences: Open User Settings (JSON)
Final newline	"files.insertFinalNewline": true
Trim	"files.trimTrailingWhitespace": true
Linux repos	"files.eol": "\n"
Diffs	"diffEditor.ignoreTrimWhitespace": false
Rulers	"editor.rulers": [80, 100, 120]
Whitespace	"editor.renderWhitespace": "boundary"
Terminal	"terminal.integrated.defaultProfile.windows": "Git Bash" or "PowerShell"
Noisy auth	Consider "git.autofetch": false on locked-down/air-gapped networks.

Extensions to Mirror Offline

Gate first	Only approved .vsix files; pin versions; document hash/source.
PowerShell	Editing, IntelliSense, formatting, script checks
YAML	Schema hints, indentation help
Ansible	Playbook editing where approved
ShellCheck/shfmt	Shell lint/format hooks where binaries exist
Markdownlint	Runbooks, SOPs, README hygiene

File-Type Notes

YAML	Spaces only. Watch indentation. Validate before running tooling.
Bash	Prefer shellcheck/shfmt. Run bash -n before commit.
PowerShell	Run as intended user/admin context; terminal elevation matters.
Kickstart	Treat %pre/%post as shell scripts; validate escaping separately.
systemd	Verify units; be explicit about restart behavior.
Markdown	Preview for runbooks, SOPs, READMEs, handoff notes.

Troubleshooting Checklist

Git auth loops	Open Git Output; check credential helper, SSH key, remote URL, autofetch.
Wrong line endings	Click CRLF/LF in status bar. Use LF for Linux-controlled files.
Format broke file	Undo; format selection only; config formats are not all code.
Shortcut weirdness	Use Keyboard Shortcuts Troubleshooting; extensions change contexts.
Remote weirdness	Try plain ssh first. If ssh is noisy/broken, VS Code Remote will be too.
Dirty repo	git status; git diff; git restore --staged file; git restore file - know what each discards.

Snippets Worth Making

systemd	service/timer skeletons with hardened defaults
repo	.repo stanza with baseurl, gpgcheck, enabled, priority
Kickstart	%post logging wrapper and common package blocks
Ansible	task/block/rescue template and handler template
PowerShell	param block, transcript/log wrapper, error handling
Runbooks	Purpose, impact, rollback, validation, owner